

Class, Object, Constructors, Static Concepts, Access Modifiers, Properties — Clean Structured Notes

1. Class and Object — Core Concepts

Class (User-Defined Data Type)

A class groups related **data (fields)** and **behaviour (methods)** into a single logical unit.

Each object created from a class represents one complete record.

This prevents data from getting scattered across multiple unrelated variables.

Why Class Is Better Than Parallel Arrays

If multiple arrays are used (e.g., one for names, one for salaries):

- Sorting or modifying one array breaks the mapping.
- Debugging becomes difficult.
- Code becomes fragile and error-prone.

With a class:

- You sort or modify complete records, not individual fields.
- Data remains consistent and logically grouped.
- Maintenance becomes easier and defects reduce.

Memory Behaviour

- **Reference (object variable)** lives on the stack.
- **Actual object instance** lives on the heap.
- Each object has its **own copy of instance fields**.



2. Constructor — Core Concept

Definition

A constructor:

- Has the same name as the class.
- Has no return type.
- Executes automatically when an object is created.
- Initializes instance variables.

Default / No-Argument Constructor

- Provided automatically **only when you do NOT create any constructor**.
- You may also define it manually.
- Access should allow object creation from outside when needed (public or internal).

Parameterised Constructor

- Receives values and initializes fields.
- When you define one, **the compiler does NOT provide a default constructor.**

Rules

- If only parameterized constructors exist → new Employee() is not valid.
- Constructor access level must allow object creation.
- Constructor runs before the object reference becomes usable.

Why Parameterised Constructors Are Needed (DRY Principle)

They:

- Avoid repeated manual assignments such as obj.Name =
- Are essential when fields are private.
- Guarantee that every object is correctly initialized when created.

3. new Keyword — Purpose

- Employee obj;
Reference exists, but it points to nothing (null).
- obj = new Employee(...);
Memory is allocated on heap and reference is linked.

Only after new can the reference safely access object members.

4. Shallow Copy — Practical Concept

Example:

```
Employee obj2 = obj1;
```

Meaning:

- No new object is created.
- Both references point to the SAME object.
- Any modification through one reference affects the same object.

Equality

For reference types (unless overridden):

- == compares reference identity.
- Equals() compares reference identity.

Contrast With Value Types

```
int b = a;
```

Value is copied — not the reference.

5. Good Practice Summary

- Model real-world entities using classes (Employee, Student, Account, etc.).
 - Use constructors to guarantee initialization.
 - Minimize repetitive logic.
 - Understand reference vs memory behaviour.
 - Avoid unintended shallow copying.
-

6. Conceptual Flow

1. Class defines structure.
 2. Object represents actual data.
 3. `new` allocates memory.
 4. Constructor initializes data.
 5. Reference points to heap memory.
 6. Multiple references may point to the same object.
- 
-

7. Instance Methods

- Operate on instance members.
 - Called via object reference
Example: `emp.Calculate()`.
-

8. Memory & Method Call Flow

- Reference → Stack
- Object Data → Heap
- Class Metadata & Method Addresses → Method Table

When calling:

```
emp.Calculate()
```

The method accesses instance data and returns results.

9. Access Modifiers — Key Points

- Default member access = private.
- Default top-level class access = internal.
- Allowed for top-level class = internal or public.

To access a method externally → define it as public or internal.

Data hiding is achieved using **private fields**.

10. Constructor Overloading

Definition

Multiple constructors with different parameter lists allow objects to be created in different ways.

Rules

- Same name as class.
- No return type.
- Signatures must differ.
- If any parameter constructor exists → no default constructor is provided automatically.
- If none exists → compiler provides a default constructor.

Why Needed

Allows object creation with different data availability:

```
new Employee("Raj","Shah","Dhirubhai");
```

```
new Employee("Naina","Chako");
```

this Keyword — Uses

this refers to the current object and is used to:

1. Differentiate field vs parameter names.
2. Chain constructors.
3. Pass current object as argument.

Example:

```
this.id = id;
```

Incorrect Assignment Example

```
salary = salary;
```

No effect — assigns variable to itself.

11. Constructor Chaining (this())

Used to centralize initialization and eliminate duplication.

Example pattern:

```
public Customer(string fn): this(fn,"",") {}
```

```
public Customer(string fn,string ln): this(fn,ln,"") {}
```

Advantages

- Single validation point
- Cleaner structure
- No duplicate logic
- Easier maintenance

12. Private Constructor, Singleton, Factory Pattern

Private Constructor

- Prevents external object creation.
- Object must be created inside the class.

Singleton Pattern

Ensures **only one object exists globally**.

Key elements:

- Private constructor
- Static reference to object
- Public static accessor method

Used for:

- Logging
- Config managers
- Connection pools
- Cache
- Shared resources

Factory Pattern

Object created through a static method instead of direct new.

Difference from Singleton:

- Factory may return new objects.



- Singleton always returns the same object.
-

13. Static Constructor

Executed **once per class**, automatically when class is loaded.

Purpose

Initialize static data or perform one-time setup.

Rules

- No access modifier
- No parameters
- Only one per class
- Runs before first object or static member access
- Cannot access instance members
- Cannot use this

Execution Flow

- Class loads
 - Static constructor runs (once)
 - Instance constructors run for each object
-



14. Static Variables (Class Level)

- Belong to class — not objects.
- Shared across all instances.
- Accessed using `ClassName.Variable`.
- Initialized once.

Used for:

- Interest rates
 - Tax rate
 - Counters
 - Common configuration
-

15. Static Class

Characteristics:

- Contains only static members.
- Cannot be instantiated.
- Implicitly sealed.
- Often used for utility logic.

Examples:

- Console
 - Math
-

16. const Keyword (Compile-Time Constant)

Characteristics:

- Must be initialized at declaration.
- Value never changes.
- Resolved at compile time.
- Implicitly static.

Use when value:

- Is fixed permanently
 - Is known at compile time
-



17. readonly Keyword (Runtime Constant)

Characteristics:

- Value assigned once.
- Assignment allowed at declaration or constructor.
- For static readonly → assign in static constructor.

Use when:

- Value comes from runtime source
 - Must not change after initialization
-

18. Access Modifiers — Full Summary

Modifier	Scope
private	Within class only
protected	Class + derived classes
internal	Same assembly only
protected internal	Same assembly OR derived class elsewhere
public	Everywhere

Top-level classes can be only:

- public
- internal

19. Properties — Core Concept

Properties provide controlled access to private fields using get and set.

Basic Form

```
public double Salary
{
    get { return _salary; }
    set { _salary = value; }
}
```



Why Use Properties

- Maintain data-hiding
- Add validation logic
- Prevent uncontrolled modification

Validation Example

```
if (value.Length < 3)
    throw new Exception(...)
```

Read-Only Property

Define only get.

Write-Only Property

Define only set (rarely used).

Auto-Implemented Properties

```
public string Name { get; set; }
```

Indexers in C#

1. Definition

An **Indexer** allows objects of a class to be accessed like arrays using [] notation. They enable index-based access to class members without exposing fields directly.

Example intent

```
Employee emp = new Employee(101,"Vita");
```

```
Console.WriteLine(emp[0]); // prints ID
```

```
Console.WriteLine(emp[1]); // prints Name
```

```
emp[0] = 202;           // modifies ID
```

This is not possible unless an indexer is defined inside the class.

2. Syntax of an Indexer

```
public object this[int index]
{
    get
    {
        // return value based on index
    }
    set
    {
        // assign value based on index
    }
}
```



3. Meaning of Each Part

Part	Meaning
public	Access modifier
object	Return type (object used because it supports any data type)
this	Indicates indexer belongs to the current class
[int index]	Parameter used as index
get	Returns value
set	Assigns value

4. Practical Example (Employee Class)

```
namespace IndexersDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }

        public Employee(int ID, string Name)
        {
            this.ID = ID;
            this.Name = Name;
        }

        public object this[int index]
        {
            get
            {
                if (index == 0)
                    return ID;
                else if (index == 1)
                    return Name;
                else
                    return null;
            }
            set
            {
                if (index == 0)
                    ID = Convert.ToInt32(value);
                else if (index == 1)
                    Name = value.ToString();
            }
        }
    }
}
```



5. Usage

```
Employee emp = new Employee(101, "Vita");
```

```
Console.WriteLine("EID = " + emp[0]);
```

```
Console.WriteLine("Name = " + emp[1]);
```

```
emp[0] = 202;
```

6. Key Points

- Indexers make objects act like arrays
 - You can read/write values using index position
 - get returns value
 - set assigns value
 - Typically return type is object so both numbers and strings can be stored
 - Indexer name is always this
 - A class may define multiple indexers with different parameter types
-

7. When To Use Indexers

Use indexers when:

- ✓ The class logically represents a **collection of values**
 - ✓ You want **array-style access without exposing fields**
 - ✓ Examples: employee lists, product lists, book catalogs, etc.
-

8. Difference Between Property and Indexer

Property	Indexer
Accessed by name	Accessed using index []
One property per name	Can map multiple fields to index values
Example: emp.Name	Example: emp[1]

9. Constraints and Good Practices

- Avoid indexers if property names give better clarity
 - Validate index range to avoid runtime errors
 - Prefer strong types over object when index always maps to same type
 - Use only when logically meaningful (not for random fields)
-

Static Methods in C#

1. When to Declare a Static Method

- Use when a task must be performed frequently and is **not tied to a specific object**
 - Suitable for **utility / helper logic**
-

2. How to Call a Static Method

- ClassName.MethodName()
 - No object creation is required
-

3. Where Static Methods Reside

- Stored in a **separate method memory block (not inside object / heap)**
 - Loaded once and shared for the entire application
-

4. Performance Note

- Static methods execute faster than instance methods
 - Because they are **not looked up through an object**
 - Remain in memory until the application closes
-

5. Static Methods in .NET Framework — Examples

- `Array.Sort()`
- `Array.BinarySearch()`
- `Console.WriteLine()`
- `Console.ReadLine()`

All of the above are called using the class name.

6. Basic Example

class Calculator

```
{  
    public static double Add(double a, double b)  
    {  
        return a + b;  
    }  
}
```

```
    public static double Sqr(double a)  
    {  
        return a * a;  
    }  
}
```

Usage

```
double x = Calculator.Add(4, 9);
```

```
double y = Calculator.Sqr(5.5);
```

7. Static Method Working with Instance Data (Through Reference)

class Employee

```
{  
    int id;  
    string name;  
    double salary;
```

```
public Employee(int id, string name, double sal)
{
    this.id = id;
    this.name = name;
    salary = sal;
}

public static double PayTax(Employee obj)
{
    return obj.salary * 0.1;
}
}
```

Usage

```
Employee e1 = new Employee(1,"Raj",5000);

double tax = Employee.PayTax(e1);
```

Key idea

Static methods **cannot access instance members directly**, but they can access them through an object reference.

8. Restrictions on Static Methods

- No this reference (because they do not belong to any object)
 - Can call only **static members directly**
 - Can access only **static data directly**
 - Instance data requires an object reference
-

9. Important Clarifications

- You can create an object inside a static method
(Example: objects created in Main())
- A static method **can call an instance method**, but only via object reference
- A static method **cannot work with instance variables directly**

Correct Example

```
public static double PayTax(Employee obj)
{
    return obj.salary * 0.1;
}
```

Incorrect Example (Not Allowed)

```
public static double PayTax()
{
    return salary * 0.1; // ERROR
}
```

Arrays in C#

Definition

An array is a collection of elements of the same data type stored in contiguous memory locations. Arrays are reference types, and all arrays in C# are derived from the abstract class System.Array.

Declaration and Initialization

```
int[] a = new int[3];
```

```
a[0] = 100;
```

```
a[1] = 200;
```

```
a[2] = 300;
```

Key Points

- Array size must be specified at creation
 - Elements are accessed using zero-based indexing
 - Length is a read-only property from the parent Array class
-

Declaration and Initialization in One Line

```
string[] s = new string[] { "one", "two", "three" };
```

```
int[] x = new int[4] { 20, 22, 23, 0 };
```

Implicitly Typed (Anonymous) Array

```
var b = new[] { 1, 1.5, 2, 2.5 }; // type becomes double[]
```

Object Array (Generally Not Recommended)

An object[] can store mixed data because all types derive from object.

```
object[] data = new object[4];
```

```
data[0] = 10;
```

```
data[1] = false;
```

```
data[2] = new DateTime(1969, 3, 24);
```

```
data[3] = "Form & Void";
```

Accept Size from User and Sum Elements (Concept Summary)

- Read size
 - Create array
 - Read elements
 - Sum values
-

Two-Dimensional Array (Rectangular Array)

```
int[,] table = new int[3, 3];
```

```
table[0,0] = 1;
```

Represents a matrix of rows and columns.

Jagged Array (Array of Arrays)

```
int[][] jag = new int[3][];
```

```
jag[0] = new int[4];
```

```
jag[1] = new int[5];
```

```
jag[2] = new int[6];
```

Each row may have a different length.

Passing Array to a Method

Arrays are passed by reference, so changes inside the method affect the original array.

Important Array Class Methods

- CopyTo() → copies elements into another array
 - Sort() → sorts array elements
 - Reverse() → reverses order fully or partially
-

Array of Objects

You can store objects (for example Employee objects) inside an array and iterate to display values.

Common Array-Based Tasks (Concepts)

- Check whether data is ascending, descending, or unsorted
 - Search for a name in a list of names
 - Count word occurrences in a sentence
 - Separate negative and positive numbers into a new array
 - Check substring repetition
-

Core Examination Points

- Array is a fixed-size homogeneous collection
 - Index starts at 0
 - Arrays are reference types
 - All arrays derive from System.Array
 - Length is used to get size
 - Jagged array is not the same as multidimensional array
 - Arrays passed to methods exhibit reference behavior
-

Inheritance in C#

Definition

Inheritance allows a derived (child) class to reuse and extend the data and behavior of a base (parent) class.

Key Points

- Every class in C# is ultimately derived from System.Object
 - ToString(), Equals(), and GetHashCode() come from Object
 - Syntax for inheritance: class Child : Parent { }
 - The is operator checks type compatibility
-

Access to Parent Members

- Public members are accessible in child classes
 - Private members are inherited but not directly accessible
 - use public properties (get/set) to expose safely
 - Protected members are accessible inside child classes
-

Example — Simple Inheritance

```
class TwoDShape
{
    public double Width { get; set; }
    public double Height { get; set; }

    public void ShowDim()
    {
        Console.WriteLine("Width = " + Width + ", Height = " + Height);
    }
}

class Triangle : TwoDShape
{
    public string Style;

    public double Area()
```



```
{  
    return Width * Height / 2;  
}  
}
```

Explanation:

Triangle can use Width, Height, and ShowDim() due to inheritance.

Constructors and base Keyword

- When you create a child object, parent constructor executes first
- If the parent has a parameterized constructor, it must be called using base()

```
class TwoDShape  
{  
    public TwoDShape(double w, double h)  
    {  
        Width = w;  
        Height = h;  
    }  
}
```

```
class Triangle : TwoDShape  
{  
    string Style;  
  
    public Triangle(string s, double w, double h)  
        : base(w, h)  
    {  
        Style = s;  
    }  
}
```



If the parent does not have a parameterless constructor and you do not call base(...), the code will not compile.

Important Concepts

- C# supports single inheritance only
 - Parent object is not created separately; only child object exists, initialized via constructor chaining
 - Inheritance is used to reuse common features and add or modify what is different
-

Object Class Inheritance

Every class inherits:

- ToString()
- Equals()
- GetHashCode()

So calling obj.ToString() is always valid.

Bank Account Example (Concept Summary)

Parent class: Account

- Fields: Id (auto-increment), Name, Balance
- Id has getter only
- Balance has protected setter
- Deposit() increases balance
- Static constructor prints bank name
- ToString() displays details

```
class Account
{
    static int idCounter;

    public int Id { get; }

    public string Name { get; set; }

    public double Balance { get; protected set; }

    public Account(string name, double bal)
    {
        Id = ++idCounter;
        Name = name;
        Balance = bal;
    }

    public void Deposit(double a)
    {
        Balance += a;
    }
}
```

Child class: SavingAccount

- Constant minimum balance = 1000
- Withdraw allowed only if balance stays ≥ 1000

```
class SavingAccount : Account
{
    const double MinBal = 1000;

    public SavingAccount(string n, double b) : base(n, b) { }
```



```

public void Withdraw(double a)
{
    if (Balance - a < MinBal)
        throw new Exception("Balance cannot be less than 1000");
    Balance -= a;
}
}

```

Child class: CurrentAccount

- Negative balance allowed

```

class CurrentAccount : Account
{
    public CurrentAccount(string n, double b) : base(n, b) {}

    public void Withdraw(double a)
    {
        Balance -= a;
    }
}

```

Key learning from example

- Common features are implemented in Account
- SavingAccount and CurrentAccount implement different withdrawal rules
- Protected setter prevents changing balance from outside classes



Constructor Calling Order in Inheritance

When you create an object of a derived class, constructors are called from parent to child.

Concept example

```

class A { public A(){ Console.WriteLine("A"); }}
class B : A { public B(){ Console.WriteLine("B"); }}
class C : B { public C(){ Console.WriteLine("C"); }}

```

Creating new C() prints:

A
B
C

Parent constructor executes first, then child.

Constructors with Parameters and base

If a parent class has a parameterized constructor, the child must call it using `base(...)`.

```
class A
{
    public A(){ }
```

```
class B : A
{
    public B(int x) : base()
    {
    }
}
```

Important rules:

- If you do not write `base(...)`, the parameterless parent constructor is called automatically
 - If the parent has no parameterless constructor, you must call the parameterized one
 - If constructor chaining is incorrect, the code will not compile
-

Execution Flow

When a child object is created:

- Parent constructor runs first
 - Then child constructor runs
 - Initialization moves from top to bottom along the inheritance chain
-

base Keyword

`base` is used to:

- Call parent constructor
- Access parent members hidden by child

```
public Triangle(string s, double w, double h) : base(w, h)
{
}
```

Inheritance and Access Modifiers

Class accessibility rule

A child class cannot be more accessible than its parent.

Example producing an error:

```
class Account {}  
  
public class Current : Account { } // ERROR
```

Correct:

```
public class Account {}  
  
class Current : Account { } // OK
```

Access Modifiers — Key Points

Class accessibility (assembly level)

- public → accessible outside assembly
- internal → accessible only inside assembly

Member accessibility

- public → accessible everywhere
 - private → accessible only inside the same class
 - protected → accessible in the same class and child classes
 - protected internal → accessible in same assembly or child classes in other assemblies
 - private protected → accessible only inside same assembly and inside derived classes
-

Important Clarifications

- To access a method outside an assembly, the class and method must both be public
 - A public method inside an internal class is not accessible outside the assembly
 - Protected members are not accessible using an object, only via inheritance
-

Constructor and Inheritance Summary

- Constructors are not inherited
 - Parent constructor always runs first
 - base(...) is used to call parameterized parent constructor
 - Child class cannot have higher accessibility than parent
 - Protected members are available to derived classes
-

Method Hiding in C#

Definition

Method hiding occurs when a derived (child) class declares a member (method or variable) that has the same name as a member in the base (parent) class. The child's version hides the base version inside the child class. This is **not polymorphism** and **not overriding**.

To indicate that hiding is intentional, the new keyword is used. If new is not used, the program still works, but the compiler issues a warning.

Example — Variable and Method Hiding

```
class A
{
    public int i = 0;

    public void show()
    {
        Console.WriteLine("i in base class: " + i);
    }
}

class B : A
{
    new int i; // hides A.i

    public B(int a, int b)
    {
        base.i = a; // assigns parent variable
        i = b;      // assigns child variable
    }

    public new void show()
    {
        base.show(); // calls base class show()
        Console.WriteLine("i in derived class: " + i);
    }
}
```

Usage:

```
B ob = new B(1, 2);
```

```
ob.show();
```

Output:

```
i in base class: 1
```

```
i in derived class: 2
```

So two separate variables exist:

Member	Value
base.i	1
child i	2

This proves that method hiding isolates parent and child members — they do not override each other.

Important Rule

If a child intentionally hides a base class method, declare it as:

```
public new void show()
```

If new is omitted, the compiler still hides the base method but shows a warning because hiding may be accidental.

Why Method Hiding Is Needed

Sometimes you inherit a class from a third-party library. Later, the library updates and adds a method with the same name as one of your methods.

To avoid breaking your code and still preserve your earlier behavior, you can hide the new library method using new.

Using base in Method Hiding

Even when the child hides a method, the parent implementation is still available:

```
base.show();
```

Parent variables can also be accessed using:

```
base.i;
```

This helps when you want to use both parent and child implementations together.

Method Hiding and Upcasting

```
A aref = new B(5, 3);
```

```
aref.show();
```

Output:

```
i in base class: 5
```

Explanation:

- The reference type is A
- So the compiler binds the call to A.show()
- Method hiding works based on **reference type**
- The base class does not know about the hidden child method

This is called **upcasting**.

Key Concepts

- Method hiding happens when base and child members share the same name
- The new keyword explicitly marks hiding
- Hiding is not overriding
- Method resolution depends on the **reference type**, not the object type
- The base keyword allows access to the hidden parent version
- When upcasting, the base class version executes

Hiding vs Overriding — Quick Comparison

Feature	Hiding (new)	Overriding (override)
Applies to	Static and instance methods	Instance methods only
virtual required?	No	Yes
Binding time	Compile-time	Runtime
Depends on	Reference type	Object type

Reference Compatibility (Type Safety)

Two references must be type-compatible.

```
class Animal { }
```

```
class Human { }
```

```
Animal a = new Animal();
```

```
Human h = new Human();
```

```
a = h; // Compile-time error
```

Reason: Human is not an Animal unless it inherits from it.

This is the same reason the following is illegal:

```
bool b = true;
```

```
string s;
```

```
s = b; // Not allowed
```

C# enforces strict type safety to prevent runtime failures.

Runtime Polymorphism (Method Overriding)

By default, C# methods are non-virtual. To enable runtime polymorphism:

- The parent method must be marked virtual
- The child method must be marked override

At runtime, the version executed depends on the **object type**, not the reference type.

Example of Runtime Polymorphism

```
class shape
{
    public int H, W;

    public shape(int h, int w) { H = h; W = w; }

    public virtual double area() { return 0; }
}

class rectangle : shape
{
    public rectangle(int p, int q) : base(p, q) {}

    public override double area()
    {
        return H * W;
    }
}

class triangle : shape
{
    public triangle(int p, int q) : base(p, q) {}

    public override double area()
    {
        return (H * W) / 2.0;
    }
}
```

Usage:

```
shape s1 = new triangle(5, 5);
Console.WriteLine(s1.area()); // 12.5
```

```
s1 = new rectangle(4, 2);
Console.WriteLine(s1.area()); // 8
```

This works because the overridden method is resolved **at runtime**.



Key Understanding — Upcasting in Polymorphism

A parent reference can point to a child object.

The parent reference exposes only parent members, but overridden methods still execute based on the actual object.

This is runtime polymorphism.

Rules for Runtime Polymorphism

- virtual enables overriding
- override replaces parent implementation
- Private members cannot be virtual or overridden
- Method resolution occurs at runtime

If the child does not override a virtual method, the parent version executes.

Polymorphism via Methods Accepting Parent Type

```
class showpoly
{
    public void call(shape s)
    {
        Console.WriteLine(s.area());
    }
}
```

Calling:

```
triangle t = new triangle(5, 5);
rectangle r = new rectangle(5, 5);
```

```
showpoly p = new showpoly();
p.call(t);
p.call(r);
```

The correct overridden method executes every time.

ToString() and Polymorphism

ToString() is virtual in System.Object, so it can be overridden.

```
public override string ToString()
{
    return id + " " + name + " " + salary;
}
```

When you print an object, Console.WriteLine(obj) internally calls ToString().

If ToString() is not overridden, the default output is:

Namespace.ClassName

Overriding vs Hiding — Summary

Feature	Overriding	Hiding
Keyword	override	new
Parent method must be	virtual	anything
Binding	Runtime	Compile-time
Depends on	Object type	Reference type

Abstract Methods and Polymorphism

An abstract method forces child classes to override it.

abstract class Account

```
{  
    public abstract void Withdraw();  
}
```

Key properties:

- Abstract methods have no body
- The class must be declared abstract
- All derived classes must override the abstract method

This guarantees consistent behavior design.

Abstract Classes — Key Facts

- Abstract class cannot be instantiated
 - It may contain abstract or non-abstract methods
 - Abstract classes can have constructors
 - Arrays and ValueType are abstract classes in .NET
 - Members of an abstract class are accessed via derived classes
-

Virtual vs Abstract Method — Difference

- Virtual method → override is optional
 - Abstract method → override is mandatory
 - Class with abstract method must be abstract
 - You can instantiate a class with only virtual methods, but not with abstract methods
-

Practical Use Case (Bank Accounts)

An abstract Account class can define:

- Common members → Id, Name, Balance
- Common method → Deposit
- Abstract method → Withdraw

SavingAccount:

- Must maintain minimum balance = 1000

CurrentAccount:

- Allows negative balance

Common features remain in parent; unique features go into child classes.

Method Sealing

A sealed method cannot be overridden further.

```
public sealed override double Area() { }
```

Use this to lock method behavior in child classes so deeper inheritance cannot change critical functionality.

Sealed Class

A sealed class cannot be inherited.

This is used when behavior must not be modified.

Examples in .NET:

- String
 - StringBuilder
 - OperatingSystem
 - SqlConnection
-

Printing an Object and Object Methods

- Console.WriteLine(obj) calls obj.ToString() internally
- Equals() checks object equality
- ReferenceEquals() checks reference (memory) equality
- GetHashCode() supports hashing collections

If overriding Equals(), you must override GetHashCode() as well — for correctness in dictionary-based collections.

Reference vs Value Equality

If two references point to the same object:

- == → true
- Equals() → true
- ReferenceEquals() → true
- GetHashCode() → same value

But two different objects with the same data are not equal unless Equals() and GetHashCode() are overridden.

Safe Equals Override Pattern

```
public override bool Equals(object obj)
{
    if (obj is Person temp)
        return temp.fnm == fnm && temp.lnm == lnm && temp.age == age;
```

```
    return false;
}
```

This ensures:

- Type-safe comparison
 - Null-safety
 - Logical value equality
-

Interfaces in C#

Concept

- An interface defines **what must be done, not how it is done**.
- It contains only member declarations (signatures), not implementations.
- Any class or struct implementing an interface must implement **all declared members**.
- Interfaces cannot be instantiated directly.
- A class can inherit one base class but may implement **multiple interfaces**.
- All interface members are **implicitly public** and cannot have access modifiers.
- Interfaces support runtime polymorphism through interface references.

Example idea:

IMessageService declares SendMessage(). Different classes such as Email and Sms can implement that method differently.

Basic Interface Example

```
interface IMessageService
{
    void SendMessage(string address);
}

class Email : IMessageService
{
    public void SendMessage(string address)
    {
        Console.WriteLine($"Sending Email to {address}");
    }
}

class Sms : IMessageService
{
    public void SendMessage(string address)
    {
        Console.WriteLine($"Sending Sms to {address}");
    }
}
```

Usage:

```
IMessageService s = new Sms();

s.SendMessage("Vita");
```

This demonstrates runtime polymorphism because the **object decides implementation**, not the reference.

Interface Reference Pointing to Implementing Class

```
IMessageService obj = new Email();
```

```
obj.SendMessage("DBDA");
```

The actual method executed depends on the object at runtime.

Static Helper Function Working with Interfaces

```
static class MessageProvider
{
    public static void Send(IMessageService s, string add)
    {
        s.SendMessage(add);
    }
}
```

Passing different objects results in different behavior while keeping one API.

When Parent Class Also Has Same Method Name

If the parent class contains a method with the same signature as the interface, the child method hides the parent version. The compiler warns unless you explicitly use new.

Public vs Explicit Interface Implementation

- Interface methods are normally implemented as public methods.
- You can also implement them explicitly so that the method is only callable through the interface reference.

Example:

```
interface I1 { void A(); }
```

```
interface I2 { void A(); }
```

```
class C : I1, I2
```

```
{
    public void A()
    {
        Console.WriteLine("C.A()");
    }
}
```

```
void I1.A()
{
    Console.WriteLine("I1.A()");
}
}
```

Usage:

```
C c = new C();  
c.A();    // Calls C.A()
```

```
I1 i1 = c;  
i1.A();   // Calls I1.A()
```

```
I2 i2 = c;  
i2.A();   // Calls C.A()
```

Explicit interface members are hidden unless accessed through the interface type.
This helps avoid ambiguity when two interfaces define the same method name.

Important Built-In .NET Interfaces

- ICloneable — cloning objects
- IComparable — natural sorting
- IComparer — custom sorting logic
- IEnumerable — enables foreach
- IEnumerator — supports enumeration internally

ICloneable — Shallow Copy Using MemberwiseClone

```
public class Point : ICloneable  
{  
    public int X { get; set; }  
    public int Y { get; set; }  
  
    public object Clone()  
    {  
        return this.MemberwiseClone(); // shallow copy  
    }  
}
```

Shallow copy duplicates the object but **nested reference fields still point to the same instance**.

Deep Copy Pattern

```
public object Clone()  
{  
    Point newPoint = (Point)this.MemberwiseClone();  
    newPoint.desc = new PointDescription();  
    newPoint.desc.PetName = this.desc.PetName;  
    return newPoint;  
}
```

Deep copy creates separate inner objects too.

IComparer — External Sorting Logic

```
class DataSort : IComparer
{
    public int Compare(object x, object y)
    {
        employee t1 = x as employee;
        employee t2 = y as employee;
        return String.Compare(t1.Name, t2.Name);
    }
}
```

Usage:

```
Array.Sort(p, new DataSort());
```

IComparable — Natural Sorting

```
class employee : IComparable
{
    public float Salary { get; set; }

    public int CompareTo(object obj)
    {
        employee temp = obj as employee;
        return this.Salary.CompareTo(temp.Salary);
    }
}
```

Usage:

```
Array.Sort(p);
```

IEnumerable — Enables foreach

```
class company : IEnumerable
{
    employee[] e = new employee[5];

    public IEnumerator GetEnumerator()
    {
        return e.GetEnumerator();
    }
}
```

Usage:

```
foreach (employee ep in cp)
    Console.WriteLine(ep.Name);
```

IEnumerator — Custom Enumerator Pattern

```
class ProductEnumerator : IEnumerator
{
    int index = -1;
    string[] ProductList;

    public object Current => ProductList[index];

    public bool MoveNext()
    {
        index++;
        return index < ProductList.Length;
    }

    public void Reset()
    {
        index = -1;
    }
}
```



C# 8 — Default Interface Methods

Before C# 8, interfaces could only declare members.

From C# 8 onward, interfaces can include **implemented members** (default methods).

This allows adding new functionality to interfaces **without breaking existing implementations**.

Key Rules

- Interfaces can now contain implemented methods, properties, events, and indexers
- Supported access modifiers include: private, protected, internal, public, static, virtual, abstract, sealed
- Static members are allowed
- Default access remains public
- Default interface methods are accessible **only through interface references**
- Overriding is optional
- Overloading is supported

Purpose: share reusable behavior across unrelated classes.

Basic Example — Default Interface Method

```
interface I_interface
{
    void display_1();

    public void display_2()
    {
        Console.WriteLine("Hello!! Default Method");
    }
}
```



```
class Example_Class : I_interface
{
    public void display_1()
    {
        Console.WriteLine("Hello!! Method");
    }
}
```

Usage:

```
Example_Class t = new Example_Class();
```

```
t.display_1();
```

```
I_interface obj = t;
```

```
obj.display_2();
```

Note: display_2() must be called using the interface reference.

Overriding Default Method in Derived Interface

```
interface I_interface
{
    void display_1();

    public void display_2()
    {
        Console.WriteLine("Default Method of I_interface");
    }
}
```

```
interface A_interface : I_interface
{
    void I_interface.display_2()
    {
        Console.WriteLine("Overridden default method");
    }
}
```

A class can implement both.

Overloading Default Methods

```
interface I_interface
{
    void display_1();

    public void display_1(int a, int b)
    {
        Console.WriteLine("Sum: " + (a + b));
    }
}
```



Important Clarifications

- Default methods are accessed only via interface reference
- Classes still implement abstract members
- Multiple interfaces may override independently
- Supported in .NET Core 3.0 / .NET Standard 2.1+

Anonymous Types

Anonymous types let you create lightweight objects **without defining a class**. They are mostly used in LINQ queries.

Example:

```
var v = new { Amount = 108, Message = "Hello" };
```

- Properties are inferred and read-only
 - Type name is compiler-generated
 - They inherit from System.Object
-

Rules

- Anonymous types cannot contain methods or events
 - They are sealed
 - Created using default constructor
 - Override Equals(), GetHashCode(), and ToString()
 - Type is inferred via var
-

Reflection Example

```
var myCar = new { Color = "Pink", Make = "Saab", CurrentSpeed = 55 };
```

Calling reflection shows type and overrides.

Equality Behavior

Two anonymous values with identical structure and order are logically equal:

```
firstCar.Equals(secondCar); // true
```

```
firstCar == secondCar; // false
```

Boxing and Unboxing

Boxing converts a **value type** → **object (heap)**.

Unboxing converts **object** → **value type**.

Example:

```
int i = 123;
```

```
object o = i; // boxing
```

```
int j = (int)o; // unboxing
```

- Boxing is implicit
 - Unboxing is explicit
 - Boxing allocates heap memory, so it is expensive
-

Passing value types to methods expecting object triggers boxing.

Exception Handling in C#

Purpose

- Exceptions prevent application crashes
 - Unhandled exceptions terminate programs
 - Handling allows recovery and continuation
-

Exception Hierarchy

All exceptions derive from System.Exception.

Common CLR exceptions include:

- IndexOutOfRangeException
- NullReferenceException
- ArgumentException
- FileNotFoundException
- FormatException

Custom exceptions should be public.

Core Keywords

Keyword	Purpose
try	Wrap risky code
catch	Handle exception
throw	Raise exception
finally	Cleanup code



try-catch Rules

- catch executes only if exception occurs
 - derived exceptions should be caught before base types
 - generic catch should be used rarely
-

throw and Rethrow

```
throw new Exception();
```

```
throw; // preserves stack trace
```

finally Block

Executes always except when Environment.Exit() is called.

Used for:

- closing files
 - releasing resources
-

Exception Properties

Useful members:

- Message
 - StackTrace
 - TargetSite
 - ToString()
-

checked and unchecked

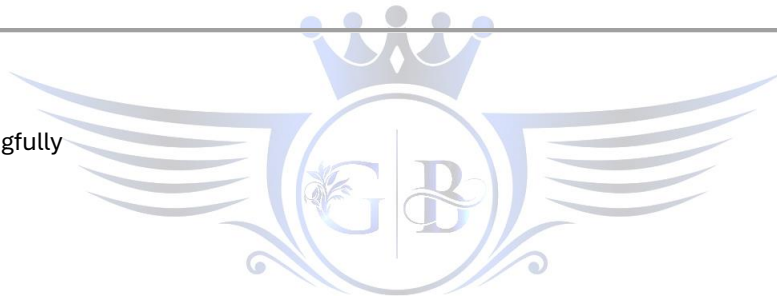
Used for overflow control.

Custom Exceptions

Create by inheriting from Exception or ApplicationException.

Best Practices

- Handle exceptions meaningfully
 - Avoid empty catch blocks
 - Use finally or using
 - Prefer specific exceptions
-



Short Example

```
try
{
    int[] a = new int[2];
    a[5] = 10;
}
catch(IndexOutOfRangeException ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    Console.WriteLine("Finished.");
}
```

C# Streams and I/O

Core Idea

A **stream** represents a flow (sequence) of bytes.

Streams allow reading/writing data from:

- files
- memory
- network sources
- other devices

Key base classes:

- Stream — base class for byte streams
 - TextReader, TextWriter — base classes for character streams
 - MarshalByRefObject — base class for remote objects (not needed in detail here)
-

When to Use Which Class

FileStream

- Works directly with raw bytes
 - Can read/write
 - single byte
 - byte array
 - Used for binary data or low-level file control (e.g., images, videos)
-

StreamReader / StreamWriter

- Handle **text (characters, strings)**
 - Provide methods such as
 - Read(), Write()
 - ReadLine(), WriteLine()
 - Use for normal text files
-

StringReader / StringWriter

- Treats **in-memory strings** like streams
 - Useful for processing text line-by-line without using files
-

BinaryReader / BinaryWriter

- Read/write primitive data in **binary format**
 - ReadBoolean()
 - ReadByte()
 - ReadInt32()
 - Write(string)
 - Write(double)

Use when:

- compact storage required
- exact data type must be preserved

FileStream Constructor — Access Control

FileStream(string path, FileMode mode, FileAccess access)

- path → file name
- mode → how the file is opened
- access → permission

FileAccess options

- Read
- Write
- ReadWrite

FileMode options (very important)

FileMode	Meaning
Append	Opens and writes at end
Create	Creates new file, overwrites existing
CreateNew	Fails if file exists
Open	Opens existing file
OpenOrCreate	Opens or creates
Truncate	Clears existing file

Important Methods (Summary)

Class	Key Methods
FileStream	ReadByte(), WriteByte()
StreamReader	ReadLine(), Read()
StreamWriter	WriteLine(), Write()
StringReader	ReadLine()
StringWriter	WriteLine()
BinaryReader	ReadInt32(), ReadDouble(), ReadBoolean()
BinaryWriter	Write(int), Write(double)

Short Examples

Writing bytes

```
FileStream fs = new FileStream("a.txt", FileMode.Create, FileAccess.Write);
fs.WriteByte((byte)'A');
fs.Close();
```

Reading bytes

```
FileStream fs = new FileStream("a.txt", FileMode.Open, FileAccess.Read);
int ch;
while((ch = fs.ReadByte()) != -1)
    Console.WriteLine((char)ch);
fs.Close();
```

Best Practice — try-catch-finally

```
FileStream fs = null;
try
{
    fs = new FileStream("a.txt", FileMode.Open);
}
catch(Exception e)
{
    Console.WriteLine(e.Message);
}
finally
{
    if(fs != null)
        fs.Close();
}
```



StreamWriter — Write Text

```
StreamWriter sw = new StreamWriter("test.txt");
sw.WriteLine("Hello");
sw.Close();
Append mode:
StreamWriter sw = new StreamWriter("test.txt", true);
```

StreamReader — Read Text

```
StreamReader sr = new StreamReader("test.txt");

string line;

while((line = sr.ReadLine()) != null)
    Console.WriteLine(line);

sr.Close();
```

BinaryReader / BinaryWriter

```
BinaryWriter bw = new BinaryWriter(new FileStream("data.bin", FileMode.Create));  
bw.Write(10);  
bw.Write(12.5);  
bw.Write(true);  
bw.Close();
```

```
BinaryReader br = new BinaryReader(new FileStream("data.bin", FileMode.Open));  
int x = br.ReadInt32();  
double y = br.ReadDouble();  
bool b = br.ReadBoolean();  
br.Close();
```

Stores data compactly in binary.

StringReader / StringWriter

Used for string-based streaming.

```
StringWriter sw = new StringWriter();  
sw.WriteLine("Hello");
```

```
StringReader sr = new StringReader(sw.ToString());
```

Advantages over StringBuilder:

- supports ReadLine() and WriteLine()
 - newline handling is OS-independent
-

File Class Shortcuts

```
TextReader tr = File.OpenText("test.txt");  
  
string data = tr.ReadToEnd();  
  
tr.Close();
```

IDisposable, Close(), Dispose(), and using

- Stream classes implement IDisposable
- Close() internally calls Dispose()
- Recommended safe pattern:

```
using(var stream = new StreamReader("a.txt"))  
{  
    // work here  
}
```

Resources are released automatically.

Delegates in C#

Meaning

A **delegate** is a type-safe function pointer — an object that stores a reference to a method and allows that method to be called indirectly.

Key concepts:

- Method signature must match delegate signature
 - The method is chosen at runtime
 - Enables callbacks and runtime binding
 - Essential for events
 - All delegates derive from System.Delegate
-

Delegate Syntax

```
delegate return_type DelegateName(parameters);
```

Example:

```
delegate string StrMod(string s);
```

This can refer to any method that accepts a string and returns a string.

Example — Creating and Using Delegates

```
delegate string StrMod(string s);
```

```
static string RemoveSpaces(string a)
```

```
{  
    string temp = "";  
    foreach(char c in a)  
        if(c != ' ')  
            temp += c;  
    return temp;  
}
```

```
StrMod op = RemoveSpaces;
```

```
string result = op("DAC DBDA");
```

Delegate assignment → no parentheses

Delegate call → parentheses

Delegates Bind Methods at Runtime

```
op = Reverse;
```

```
result = op("Hello");
```

Same delegate variable can call different logic dynamically.

Delegates With Return Type and No Parameters

```
delegate int Addition();
```

```
class Test {  
    int a,b;  
    public Test(int x,int y){ a=x; b=y; }  
    public int Add(){ return a+b; }  
}
```

```
Test t = new Test(6,6);
```

```
Addition d = t.Add;
```

```
int r = d();
```

Delegates With Parameters

```
delegate int MyMath(int a,int b);
```

```
static int Add(int a,int b){ return a+b; }
```

```
static int Product(int a,int b){ return a*b; }
```

```
MyMath op = Add;
```

```
int r = op(5,6);
```

```
op = Product;
```

```
r = op(5,6);
```

Delegates With Instance Methods

```
delegate int MyMath();
```

```
class Calc {  
    int a,b;  
    public Calc(int x,int y){ a=x; b=y; }  
    public int Add(){ return a+b; }  
    public int Product(){ return a*b; }  
}
```

Important Rules

- Method signature must match delegate
 - No brackets during assignment
 - Parentheses required when invoking
 - Delegates are type-safe
 - Delegates are runtime-bound
-

Why Delegates Matter

- Enable callbacks
 - Drive event handling
 - Allow runtime method swapping
 - Basis for anonymous methods and lambdas
-

Multicast Delegates

Meaning

A multicast delegate holds **multiple methods** in its invocation list.

All delegates derive from System.MulticastDelegate.

When invoked, **all methods execute in order**.

Methods are added and removed using:

- += add
 - -= remove
-

Return Type Rule

- If delegate returns void → all methods run normally
- If delegate has a return type → last method's value is returned

Hence multicast delegates are usually void.

Example

```
delegate void StrMod(string s);
```

Assigning multiple methods

```
StrMod op = RemoveSpaces;
```

```
op += Reverse;
```

```
op("This is a test");
```

Output:

This is a test
set a si sihT

Delegates With Return Type — Only Last Matters

```
delegate int Calc(int n);
```

Both execute, last value returned.

Multicast With Parameters

```
delegate void MyMath(int a,int b);
```

```
MyMath m = Add;  
m += Product;
```

```
m(5,6);
```

Output:

11

30

Access Each Method

```
foreach(MyDel a in d.GetInvocationList())
```

```
    Console.WriteLine(a.Invoke(3));
```

Key Points

- Delegates are type-safe
 - All delegates are multicast-capable
 - Execution order = registration order
 - Only last return value counts if return type exists
-

Practice Concepts

- Create delegates for factorial, prime check, etc.
 - Observe calling order
 - Watch return value behavior
-

Events in C#

Meaning

An **event** is a notification mechanism built on delegates.

When an event is raised (fired), **all subscribed handlers are automatically invoked.**

Events follow the **Publisher–Subscriber pattern**:

- **Publisher** — declares and raises the event
- **Subscriber** — registers event-handling methods

Event handlers are delegates.

Key Properties of Events

- Declared using the event keyword
 - Internally use **delegates (usually multicast)**
 - Subscribers attach handlers using +=
 - Handlers are removed using -=
 - Events are members of a class
 - By .NET convention, event handlers
 - return void
 - accept (object sender, EventArgs e)
-

Simple Custom Event Example

Step 1 — Declare delegate

```
delegate void MyEventHandler();
```

Step 2 — Declare event inside Publisher class

```
class MyEvent
{
    public event MyEventHandler SomeEvent;

    public void OnSomeEvent()
    {
        if (SomeEvent != null)
            SomeEvent(); // fire event
    }
}
```

Step 3 — Subscribe & Handle in Subscriber

```
class EventDemo
{
    static void handler()
    {
        Console.WriteLine("Event occurred");
    }
}
```

```
static void Main()
{
    MyEvent e = new MyEvent();
    e.SomeEvent += handler; // subscribe
    e.OnSomeEvent();      // fire
}
}
```

Why Use the event Keyword?

Without event, a delegate field:

- can be invoked from outside the class
- can be overwritten (unsafe)

With event:

- only the publisher can raise the event
- subscribers can only attach/detach handlers
- ensures encapsulation and safety

Invalid with event:

`e.SomeEvent();` // cannot invoke outside publisher

`e.SomeEvent = h;` // cannot assign

Events Support Multicast

Multiple handlers may subscribe:

`e.SomeEvent += Handler1;`

`e.SomeEvent += Handler2;`

Execution order equals subscription order.

Handlers are removed using:

`e.SomeEvent -= Handler1;`

Real-World Event Usage

Events are ideal for notifications such as:

- SMS/email alerts
- UI button clicks
- Progress reporting
- CRUD notifications
- Order tracking

Publisher raises **one event** — subscribers respond individually.

Publisher–Subscriber Example With Parameters

```
delegate void MyMath(int x, int y);
```

```
class Publisher
```

```
{  
    public event MyMath MathEvent;  
  
    public void Call(int x, int y)  
    {  
        if (MathEvent != null)  
            MathEvent(x, y);  
    }  
}
```

```
class Planning
```

```
{  
    public void Add(int x,int y){ Console.WriteLine(x + y); }  
    public void Product(int x,int y){ Console.WriteLine(x * y); }  
}
```

Usage:

```
Planning p = new Planning();  
Publisher pub = new Publisher();
```

```
pub.MathEvent += p.Add;  
pub.MathEvent += p.Product;
```

```
pub.Call(5,4);
```

Output:

```
9  
20
```



Anonymous Methods & Lambda with Events

Anonymous method subscription

```
acc.wdevent += delegate(double a,double bal,string n)
```

```
{  
    Console.WriteLine("Email sent to " + n);  
};
```

Lambda subscription

```
acc.wdevent += (a,bal,n) =>
```

```
    Console.WriteLine($"SMS: Amount {a}, Balance {bal}, Name {n}");
```

No separately-named method required.

Important Exam Rules

- Events MUST use delegates
- Only publisher can raise the event
- Subscribers attach via +=
- Typical .NET handler signature:

```
void Handler(object sender, EventArgs e)
```

- Events are usually multicast
- Always check for null before firing — or use:

```
SomeEvent?.Invoke();
```

Anonymous Functions & Lambda Expressions in C#

Why Anonymous Functions Exist

Sometimes you need a method **only to pass to a delegate**, not to call elsewhere.

Anonymous functions let you define the method inline.

C# supports:

- **Anonymous Methods (C# 2.0)**
 - **Lambda Expressions (C# 3.0 — preferred)**
-

Anonymous Methods

Syntax

```
delegate void MyDel();
```

```
MyDel d = delegate {  
    Console.WriteLine("Hello");  
};  
d();
```

With parameter

```
delegate void CountIt(int end);
```

```
CountIt c = delegate(int end)  
{  
    for(int i = 0; i <= end; i++)  
        Console.WriteLine(i);  
};
```

```
c(3);
```

With return value

```
delegate int Sum(int n);
```

```
Sum s = delegate(int n)  
{
```



```
int total = 0;
for(int i = 0; i <= n; i++)
    total += i;
return total;
};
```

Rules (Very Important)

- Anonymous methods **cannot access ref or out variables**
 - They **cannot declare local variables already defined outside**
 - They can be multicast like normal delegates
-

Lambda Expressions

Meaning

A **lambda expression** is a shorter syntax for anonymous methods.

Uses the operator:

=>

Read as “goes to”.

Expression Lambdas (single expression)

```
delegate int IntOp(int n);
```

```
IntOp square = n => n * n;
```

No return or { } required.

Statement Lambdas (multiple statements)

```
IntOp fact = n =>
```

```
{
    int r = 1;
    for(int i = 1; i <= n; i++)
        r *= i;
    return r;
};
```

Requires { } and return.

Type Inference

Parameter types are inferred:

```
IsEven check = n => n % 2 == 0;
```

Parentheses are optional for **one parameter**, required for multiple.

Common Lambda Examples

Check even

```
delegate bool IsEven(int n);
```

```
IsEven isEven = n => n % 2 == 0;
```

Add two numbers

```
delegate int Add(int x, int y);
```

```
Add sum = (x,y) => x + y;
```

Lambda With Multicast Delegate

If delegate has return type → **only last return value is received**

```
IntOp op = n => n*n;
```

```
op += n => n+1;
```

```
int r = op(3);
```

Result = 4 (from second lambda)

Delegates and Lambda Example

```
delegate int MathOp(int n);
```

```
MathOp m = n => n*n;
```

```
m = n => n*n*n;
```

Built-In Delegates (Very Common)

```
Func<int,int,int> add = (x,y) => x+y;
```

```
Predicate<int> isOdd = n => n % 2 != 0;
```

When to Use

Feature	Use Case
Named Method	Reusable logic
Anonymous Method	Short-term logic
Lambda Expression	Preferred for clarity

Key Differences

- Anonymous methods use delegate
 - Lambdas use =>
 - Expression lambdas omit return
 - Statement lambdas require return
 - Compiler infers types
 - Lambdas are heavily used in:
 - delegates
 - LINQ
 - events
-

Generics in C#

Using object as a Generic Type (Old, Unsafe Approach)

Before generics, code used object so any data type could be stored because all types inherit from object.

Example (Non-Generic Class)

```
class NonGen
{
    object ob;
    public NonGen(object o) { ob = o; }
    public object GetOb() { return ob; }
    public void ShowType() => Console.WriteLine(ob.GetType());
}
```

Usage:

```
NonGen n1 = new NonGen(102);
int v = (int)n1.GetOb(); // explicit cast needed
```

```
NonGen n2 = new NonGen("Test");
string s = (string)n2.GetOb();
```

Why this is BAD

- Requires explicit casting
- Type errors occur **at runtime, not compile-time**
- Cross-assignment is allowed but unsafe:

```
n1 = n2;
```

```
int x = (int)n1.GetOb(); // runtime exception
```

So this approach is **NOT type-safe**.

Generics — The Correct Solution

Generics introduce **type parameters**, allowing reusable code that remains type-safe.

Generic Class Syntax

```
class Gen<T>
{
    T ob;
    public Gen(T o) { ob = o; }
    public T GetOb() => ob;
    public void ShowType() => Console.WriteLine(typeof(T));
}
```

Usage:

```
Gen<int> g1 = new Gen<int>(102);
```

```
int v = g1.GetOb(); // no cast
```

```
Gen<string> g2 = new Gen<string>("Hello");
```

```
string s = g2.GetOb(); // no cast
```

Key Points

- T is replaced at compile time
- **No boxing/unboxing** for value types
- Type-safety enforced at compile-time



Why Generics Are Better (Core Benefits)

Generics provide:

- ✓ Type-safety
 - ✓ Better performance
 - ✓ Compile-time validation
 - ✓ Code reuse across data types
-

Generic Method Example — Swap Two Values

```
static class MyGenericMethods
{
    public static void Swap<T>(ref T a, ref T b)
    {
        T temp = a;
        a = b;
        b = temp;
    }
}
```

```
}
```

Works for:

- int
 - string
 - bool
 - custom types
-

Generic Delegates

Custom Generic Delegate

```
delegate void MyGenericDelegate<T>(T arg);
```

Usage:

```
MyGenericDelegate<string> d1 = s => Console.WriteLine(s);
```

```
MyGenericDelegate<int> d2 = n => Console.WriteLine(n + 1);
```

Predefined Generic Delegates (Very Important)

Predicate<T>

Returns bool

```
Predicate<int> isOdd = n => n % 2 != 0;
```

Action<T>

Returns void

```
Action<int,int> print = (x,y) => Console.WriteLine(x+y);
```

Func<T,..., TResult>

Returns any type

```
Func<int,int,int> add = (x,y) => x + y;
```

```
Func<int,int,string> sumToStr = (x,y) => (x+y).ToString();
```

Supports **up to 16 parameters**.

Generic Interfaces (Concept)

- IComparable<T> → CompareTo(T other)
- IComparer<T> → Compare(T x, T y)

Used for sorting.

When to Use Generics

Use generics when you want:

- Strong type-safety
- Reusable logic
- Compile-time checking
- Performance (no boxing/casting)

Avoid object storage because:

- Requires casting
 - Errors occur at runtime
 - Boxing/unboxing slows execution
-

Must-Remember Statements

- Introduced in .NET 2.0
 - Syntax uses <T>
 - Generics remove casting
 - Prevent runtime type errors
 - Improve performance and safety
 - Func, Action, and Predicate are predefined generic delegates
-

Classes, Objects & Memory Management in C#

Objects and References

- **A class defines a type**
- new creates an object on the **managed heap**
- new returns a **reference**, not the object itself
- Reference variables live on the **stack**, object lives on the **heap**

Example:

```
Car c = new Car();
```

c → stored on stack

Car object → stored on heap

Object Lifetime & Garbage Collection (GC)

Golden rule:

Create objects using new and let CLR free memory automatically.

An object is destroyed when **no live references exist**.

Heap uses a pointer indicating next allocation location.

What Happens When new Executes

- CLR determines memory size
- Ensures heap space exists
- Calls constructor
- Returns reference
- Moves heap pointer forward

If heap memory is low → **GC runs automatically**

When Does Garbage Collection Run?

GC runs **when heap lacks free memory**.

It removes **unreachable objects**.

Object Graph & Reachability

GC identifies **reachable vs unreachable** objects.

Unreachable objects become garbage and are deleted.

GC Generations

Objects are grouped by “age”:

Generation	Meaning
Gen 0	Newly created
Gen 1	Survived one collection
Gen 2	Long-lived

GC optimizes performance by **collecting Gen-0 most frequently**.

Forcing Garbage Collection (Rare)

You *can* force GC:

```
GC.Collect();
```

```
GC.WaitForPendingFinalizers();
```

Best practice → **Avoid forcing GC** unless absolutely necessary.

Used rarely for:

- large temporary object cleanup
 - batch processes
 - long-running services
-

Viewing GC Information

```
GC.GetTotalMemory(false);
```

```
GC.MaxGeneration;
```

```
GC.GetGeneration(obj);
```

```
GC.CollectionCount(0);
```

Destructors (Finalizers)

A destructor runs before GC deletes object.

Syntax

```
~ClassName()  
  
{  
    // cleanup code  
}
```

Rules:

- Same name with ~ prefix
- No parameters
- No return type
- Called by GC
- Used for unmanaged resource cleanup

Compiler converts destructor → Finalize()



Finalizer vs Dispose

Finalizer / Destructor	Dispose (IDisposable)
Called by GC	Called manually
Non-deterministic timing	Deterministic cleanup
Used for unmanaged resources	Can clean both
No parameters	Defined in interface

IDisposable:

```
public interface IDisposable  
{  
    void Dispose();  
}
```

Common users: Stream, DbConnection, etc.

.NET Collections

Why Collections?

Arrays have a **fixed size**, so they are not suitable when the number of elements is unknown.

.NET **collections are dynamic containers** that automatically resize when items are added or removed.

Generic collections live in:

System.Collections.Generic

Benefits of Generic Collections

- Type-safe — store only one data type
- No boxing/unboxing — faster performance
- Reusable and flexible
- Compile-time error detection

Common Generic Collections at a Glance

Collection	Indexed	Sorted	Duplicate Keys	Duplicate Values	Allows Null Key
List	Yes	No	NA	Yes	NA
Dictionary<TKey,TValue>	Key-based	No	No	Yes	No
SortedList<TKey,TValue>	Key-based	Yes	No	Yes	No
SortedDictionary<TKey,TValue>	Key-based	Yes	No	Yes	No
HashSet	No	No	NA	No	Yes
SortedSet	No	Yes	NA	No	Yes

List

Key Points

- Dynamic array
- Maintains insertion order
- Indexed access via []
- Strongly typed
- Capacity increases automatically

Example

```
List<char> lst = new List<char>();
```

```
lst.Add('C');
```

```
lst.Add('A');
```

```
lst.Add('E');
```

Capacity Behavior

Capacity grows like:

0 → 4 → 8 → 16 → ...

Removing items **does not shrink** internal capacity.

Modify / Remove

```
lst.Remove('A');
```

```
lst[0] = 'X';
```

Invalid (because list is strongly-typed):

```
lst.Add(99); // compile-time error
```

List with Objects

```
class Employee
{
    public string Name { get; set; }
    public double Salary { get; set; }
}
```

```
List<Employee> list = new List<Employee>
{
    new Employee{ Name="Raj", Salary=6000 },
    new Employee{ Name="Mona", Salary=7000 }
};
```

Searching Using Predicate

```
Employee obj = list.Find(e => e.Salary == 6000);
```

```
var result = list.FindAll(e => e.Salary >= 6000);
```

Predicate Delegate

A **Predicate** is a delegate that returns bool.

```
Predicate<Employee> p = e => e.Salary > 5000;
```

Used in:

- Find
 - FindAll
 - RemoveAll
-

SortedList<TKey,TValue>

Characteristics

- Data sorted by **key**
- Lookup by index or key
- Duplicate keys not allowed
- Values may repeat

Example

```
SortedList<string,double> sl = new SortedList<string,double>();
```

```
sl.Add("Butler",73000);
```

```
sl.Add("Frank",99000);
```

Access:

```
sl["Butler"];
```

```
sl.Keys;
```

```
sl.Values;
```

Throws exception if key exists.

Safe fetch:

```
sl.TryGetValue("abc", out var value);
```

Remove entry:

```
sl.Remove("txt");
```

Dictionary<TKey,TValue>

Characteristics

- Unordered
- Very fast lookup (hash table)
- Unique keys only
- Key cannot be null

Example

```
Dictionary<string,double> dict = new Dictionary<string,double>();
```

```
dict.Add("Raj",7000);
```

Iterate

```
foreach(var kv in dict)
```

```
    Console.WriteLine(kv.Key+" "+kv.Value);
```

SortedDictionary<TKey,TValue>

Same as Dictionary, **but sorted by key internally**.

HashSet

Characteristics

- Stores **unique values only**
- No index
- Extremely fast lookup
- Prevents duplicates automatically

Example

```
HashSet<string> hs = new HashSet<string>();
```

```
hs.Add("Java");
```

```
hs.Add("C#");
```

Remove:

```
hs.Remove("Java");
```

Remove conditionally:

```
hs.RemoveWhere(v => v.StartsWith("C"));
```

HashSet — Set Operations

Let

$A = \{A, B, C\}$

$B = \{C, D, E\}$

Union

```
A.UnionWith(B);
```

Result:

$\{A, B, C, D, E\}$

Intersection

```
A.IntersectWith(B);
```

Result:

$\{C\}$

Difference

```
A.ExceptWith(B);
```

Result = items only in A

Symmetric Difference

```
A.SymmetricExceptWith(B);
```

Result:



{A,B,D,E}

SortedSet

Same as HashSet but **sorted automatically**.

Comparable — Natural Sorting

Implement inside the class:

```
class Inventory : IComparable<Inventory>
{
    public string Name;

    public int CompareTo(Inventory obj)
        => Name.CompareTo(obj.Name);
}
```

Used automatically when calling:

```
list.Sort();
```

Comparer — Custom Sorting Strategy

Define externally:

```
class ComplInv : IComparer<Inventory>
{
    public int Compare(Inventory x, Inventory y)
        => x.Name.CompareTo(y.Name);
}
```

Apply:

```
list.Sort(new ComplInv());
```

IEnumerable — Enable foreach

Any class implementing IEnumerable can be iterated using foreach.

Must implement:

```
IEnumerator GetEnumerator();
```

Used to build **custom collection classes**.

Dictionary — Count Occurrences Example

```
string[] arr = { "A","B","A","F","F" };
```

```
Dictionary<string,int> freq = new Dictionary<string,int>();
```

```
foreach(string item in arr)
{
    if(freq.ContainsKey(item))
        freq[item]++;
    else
        freq[item]=1;
}
```

Lookup:

```
Console.WriteLine(freq["A"]);
```

Time Complexity

- Build dictionary → $O(n)$
- Lookup → $O(1)$



Nullable Types & Null Safety in C#

Value Types Cannot Be Null (By Default)

Value types such as `int`, `bool`, `double`, etc., **cannot store null**.

```
bool myBool = null; // Compile-time error
```

```
int myInt = null; // Compile-time error
```

Reference types **can be null**:

```
string myString = null; // Allowed
```

Nullable Value Types

A **nullable value type** can store either:

- its normal value
- or null

Syntax

```
int? x = null;
```

```
bool? flag = null;
```

```
double? d = 3.14;
```

```
char? c = 'a';
```

```
int?[] arr = new int?[10];
```

Meaning:

```
bool? b // values possible → true, false, null
```

Commonly used in **databases where values may be missing**.

Equivalent Syntax

```
int? a = null;
```

is same as:

```
Nullable<int> a = null;
```

Nullable Reference Types (C# 8.0)

Purpose: warn developers when they may dereference null accidentally.

Without null-warnings:

```
string s = null;
```

```
Console.WriteLine(s[0]); // Runtime NullReferenceException
```

With nullable reference types enabled:

```
string? s = null; // nullable reference
```

```
string t = "text"; // non-nullable
```

Compiler treats:

- string → must NOT be null
 - string? → allowed to be null
-

Null Safety Flow Analysis

```
string? s = null;
```

```
if (s != null)
```

```
    Console.WriteLine(s[0]); // Safe inside check
```

The compiler tracks whether a variable may be null.

Enable or Disable Per File

```
#nullable enable
```

```
#nullable disable
```

Used when modernizing older projects gradually.

Null-Coalescing Operator: ??

Provides a **default value when left side is null**.

```
string? s = null;
```

```
Console.WriteLine(s ?? "Hello"); // Output: Hello
```

If not null:

```
string? s = "Bye";
```

```
Console.WriteLine(s ?? "Hello"); // Output: Bye
```

Null-Coalescing Assignment Operator: ??=

Assigns a default only if variable is null.

```
string? s = null;
```

```
s ??= "Hello";
```

```
Console.WriteLine(s); // Hello
```

Null-Conditional Operators: ?. and ?[]

Prevents exceptions by **stopping the call when object is null**.

Property Access

```
string? s = null;
```

```
Console.WriteLine(s?.Length); // Safe — prints nothing
```

Index Access

```
string? s = "vita";
```

```
Console.WriteLine(s?[0]); // v
```

```
s = null;
```

```
Console.WriteLine(s?[0]); // Safe — prints nothing
```

Very useful for optional objects.

Partial Classes

Partial classes allow **splitting one class across multiple files**.

File 1

```
public partial class Employee
{
    public Employee(int id, string name)
    {
        Empld = id;
        FirstName = name;
    }

    public void DisplayEmployeeInfo()
    {
        Console.WriteLine(Empld + " " + FirstName + " " + LastName);
    }

    public void Save(int id, string firstName, string lastName)
    {
        Console.WriteLine("Saved!");
    }
}
```

File 2

```
public partial class Employee
{
    public int Empld { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
}
```

```
public int Age { get; set; }  
}
```

Rules

- Must be in the same namespace and assembly
 - Must have the same access modifier
 - Compiles into **one single class**
-

dynamic Keyword

Introduced in **.NET 4.0**

- Type checking happens at runtime

```
dynamic firstName = "Vita";
```

```
firstName.ToUpper(); // OK
```

```
firstName.toupper(); // Runtime error
```

Difference Summary

Feature	var	object	dynamic
Type decided	Compile time	Compile time	Runtime
Type can change later	No	Yes (cast required)	Yes
IntelliSense	Yes	Limited	None (runtime)
Allowed as parameter/return	No (locals only)	Yes	Yes

Examples

```
var a = 5;
```

```
a = "hi"; // Error
```

```
object o = 5;
```

```
o = "hi"; // OK
```

```
dynamic d = "hi";
```

```
d = 99; // OK
```

Limitation

Lambda expressions **cannot be used directly on dynamic** values.

```
dynamic a = GetDynamicObject();
```

```
a.Method(arg => Console.WriteLine(arg)); // ERROR
```

Serialization in C#

Definition

Serialization

Process of converting an object into a **byte stream** so it can be:

- Stored (file / DB / memory)
- Transmitted (network / web services)

Deserialization

Process of converting byte stream **back to object**

Requirement

Classes must be marked:

[Serializable]

Child classes DO NOT inherit this automatically — they must also be marked.

Why Serialization Is Needed

- Send objects to remote applications
 - Cross-domain / firewall transfer
 - Store user/session/security data
 - Persist object state
-

Types of Serialization in .NET

SOAP Serialization (SOAPFormatter)

SOAP = Simple Object Access Protocol

Characteristics

- XML-based
 - Platform independent
 - Works through firewalls
 - Debug-friendly
 - Serializes public + private fields
 - Includes type metadata
 - [NonSerialized] skips fields
 - Constructor NOT called when deserializing
 - Parameterless constructor NOT required
 - Not supported in .NET Core
 - Considered unsafe today
-

Binary Serialization (BinaryFormatter)

Namespace:

System.Runtime.Serialization.Formatters.Binary

Features

- Serializes public + private members
- Includes full type metadata
- Supports circular references
- Fast & efficient
- [NonSerialized] excludes fields
- No default constructor required
- Platform dependent
- **Obsolete & insecure in .NET Core**

Use JSON/XML instead.

XML Serialization (XmlSerializer)

Namespace:

System.Xml.Serialization

Characteristics

- Serializes **public fields & properties only**
- Private members ignored
- No [Serializable] needed
- Requires parameterless constructor
- Platform independent
- Stores only data — not type metadata
- Human readable



Example Output

```
<Employee>
  <Name>Raj</Name>
  <Salary>7000</Salary>
</Employee>
```

JSON Serialization (System.Text.Json)

Namespace:

System.Text.Json

System.Text.Json.Serialization

Behavior

- Serializes public properties
- Private fields ignored
- [JsonIgnore] skips properties
- Supports nullable types

Example:

```
class WeatherForecast
{
    [JsonIgnore]
    public int TemperatureCelsius { get; set; }
}
```

Benefits

- Lightweight
- Cross-platform
- Fast & safe
- Recommended

Security Guidance

Preferred & Safe

- XmlSerializer
- DataContractSerializer
- System.Text.Json

Avoid for Untrusted Data

- BinaryFormatter
- SoapFormatter
- LosFormatter
- NetDataContractSerializer
- ObjectStateFormatter

They allow unsafe polymorphic deserialization.

Important Serialization Attributes

Attribute	Meaning
[Serializable]	Enables Binary/SOAP serialization
[NonSerialized]	Skips a field
[JsonIgnore]	Skips JSON property
XmlIgnore	Skips XML property

Multithreading in C#

1. Basic Terms

Process

A **program in execution** managed by the OS.

Each process has:

- A **unique Process ID**
- Its **own memory and resources**

Thread

A **light-weight process** inside a process.

Every process has at least **one thread (Main thread)**.

Namespace for threading:

System.Threading

2. Why Use Multithreading?

Advantages

1. Keeps UI **responsive**
2. Uses CPU time efficiently while waiting for I/O
3. Allows **parallel processing** on multi-core CPUs

Disadvantages

1. Overhead of **context switching**
2. Code becomes **complex**
3. Harder to **debug and maintain**

Use multithreading only when benefits outweigh complexity.

3. Main Thread Example (Key Properties)

You can access the main thread as:

```
Thread t = Thread.CurrentThread;
```

```
t.Name = "MainThread";
```

Useful properties:

- Name
 - IsAlive
 - Priority
 - ThreadState
-

Creating Threads

1. Basic Thread Creation

```
Printer p = new Printer();  
  
Thread t = new Thread(new ThreadStart(p.PrintNumbers));  
  
t.Name = "Worker";  
  
t.Start();  
  
Or shorter:  
  
Thread t = new Thread(p.PrintNumbers);  
  
t.Start();
```

2. Sleep()

Pauses the **current thread**.

```
Thread.Sleep(2000); // 2 seconds
```

3. Passing Arguments to Threads

Use ParameterizedThreadStart:

```
Thread t = new Thread(MyMethod);  
t.Start(5);
```

```
void MyMethod(object n)  
{  
    // use (int)n  
}
```



Thread Lifecycle Helpers

1. Checking if a Thread Is Running — IsAlive

```
if(thread.IsAlive)
```

Returns **true** if thread is still executing.

2. Waiting for a Thread to Finish — Join()

```
thread.Join();
```

The calling thread **waits until the joined thread completes**.

Multiple Threads

You can create any number of threads:

```
new Thread(task1).Start();
```

```
new Thread(task2).Start();
```

```
new Thread(task3).Start();
```

All threads **share CPU time**.

Foreground vs Background Threads

Type	Behavior
Foreground	Keeps application alive until completed
Background	Terminates automatically when all foreground threads end

Default: **threads are foreground**

Convert to background:

```
thread.IsBackground = true;
```

Use background threads for **non-critical tasks**.

Synchronization (Shared Resource Protection)

Race Condition Problem

Occurs when **multiple threads access shared data simultaneously**.

Solution → **lock**

lock Statement

Syntax:

```
lock(obj)
```

```
{  
    // critical section  
}
```

Rules:

- Only one thread enters locked code at a time
- Other threads **wait**
- Lock releases when block exits
- Lock object should be **private**

Example:

```
private object locker = new object();
```

```
lock(locker)
```

```
{  
    // safe code  
}
```

Do NOT lock on this or public objects.

Monitor — Wait / Pulse / PulseAll (Conceptual Only)

Used for **thread coordination inside lock**:

Method	Meaning
Wait()	Releases lock + waits
Pulse()	Wakes one waiting thread
PulseAll()	Wakes all waiting threads

They must be used **inside a lock block**.

Key Methods Summary (Exam-Important)

Method	Purpose
Thread.Start()	Begin execution
Thread.Sleep(ms)	Pause current thread
Thread.Join()	Wait for thread to finish
IsAlive	Check status
IsBackground	Makes thread background
lock(obj)	Synchronize access

Example Concept: Why Threads Improve Responsiveness

If one thread sleeps or waits for I/O:

- CPU is free to execute another thread
 - Application **feels faster**, even if total work time is same
-